USC
UNIVERSIDADE DE SANTIAGO DE COMPOSTELA

Escola Técnica Superior de Enxeñaría (ETSE)

Sistemas Operativos II

Grado en Ingeniería Informática

Informe Técnico: Ejercicio Voluntario 3

Ejercicio 3 Opcional: Experimentación con Tiempos de Espera

Autores: [Alan Barreiro Martínez - Adriana Sánchez-Bravo Cuesta]
Asignatura: Sistemas Operativos II
Fecha: 20 de abril de 2026

Índice

1. Introducción **2**

2. Funcionamiento del código **2**

3. Ejecución **3**

4. Análisis de resultados **3**

5. Conclusiones **4**

1. Introducción

El propósito de este apartado es el de la experimentación de los distintos efectos que afectan al programa dependiendo del tiempo asignado a cada sleep.

2. Funcionamiento del código

Para este ejercicio, los cambios principales se encuentran en las funciones de los hilos. En ellas, se añadirán una serie de sleeps para forzar distintos comportamientos en el programa y observar los diferentes resultados.

En el apartado a se indica que dependiendo de la prioridad de cada item, la espera debe ser de 3, 2 o 1 segundo. Para ello, simplemente se debe cambiar el sleep aleatorio del ejercicio previo que se encuentra en la función `funcion_consumidor()` por un tiempo de espera según la prioridad del item:

```
else // s1 esta dentro de la caducidad
{
    consumidos++;
    printf('[%ld ms]\tConsumidor extrae de Prod %d (Prio %d) valor '%
       d' (%d)\n',
            get_timestamp(), it.origen, it.prioridad, it.valor,
               consumidos);

    pthread_cond_signal(&condp);
    pthread_mutex_unlock(&mutex);

    // se calcula el sleep aleatorio dependiendo de la prioridad del
       numero
    if(it.prioridad == 1) {
        sleep(3);
    }
    else if(it.prioridad == 2) {
        sleep(2);
    }
    else {
        sleep(1);
    }
    // se realizan las sumas
    consume_item(it);
}
```

Por otra parte, el apartado b del ejercicio trata de hacer el mismo experimento, pero situando los sleeps en la función funcion-productor(). Para este caso, se ha indicado un tiempo más dispar entre los distintos casos para poder tener una mayor representación de lo que sucede.

```
while (1){
    //se calcula el sleep aleatorio dependiendo de la prioridad
    if (id == 1) {
        sleep(1);
    }
    else if (id == 2) {
        sleep(3);
    }
    else{
```

```
        sleep (5);
    }
    //se bloquea el acceso al buffer
    pthread mutex lock(&mutex);
     //si el buffer esta lleno se bloquea hasta que el consumidor
        lance una senhal
    while (num elementos == N) {
        pthread_cond_wait(&condp, &mutex);
    }
//lee el numero del fichero
```

3. Ejecución

Para la ejecución del código simplemente se debe crear el ejecutable con:

```
$ gcc ej3_opcional.c -o ej3 -pthread
$ ./ej3
```

4. Análisis de resultados

Los siguientes resultados pertenecen al código del apartado a:

```
[1776618336255 ms] Productor 1 (Prio 1) mete '1' [Caducidad: 1s] (1)
[1776618336255 ms] Productor 2 (Prio 2) mete '2' [Caducidad: 2s] (2)
[1776618336255 ms] Productor 3 (Prio 3) mete '3' [Caducidad: 7s] (3)
[1776618336256 ms]      Consumidor extrae de Prod 1 (Prio 1) valor '1' (1)
[1776618337255 ms] Productor 2 (Prio 2) mete '32' [Caducidad: 9s] (4)
[1776618338256 ms] Productor 2 (Prio 2) mete '143' [Caducidad: 9s] (5)
[1776618339256 ms]      Consumidor extrae de Prod 2 (Prio 2) valor '32' (2)
[1776618341256 ms] Productor 3 (Prio 3) mete '2' [Caducidad: 8s] (6)
[1776618341256 ms]      (Caducado) Prod 2 (Prio 2) valor '2' tras 5s (Max: 2s)
[1776618341256 ms]      Consumidor extrae de Prod 2 (Prio 2) valor '143' (3)
[1776618342255 ms] Productor 1 (Prio 1) mete '2' [Caducidad: 4s] (7)
[1776618343257 ms]      Consumidor extrae de Prod 1 (Prio 1) valor '2' (4)
[1776618345256 ms] Productor 3 (Prio 3) mete '2' [Caducidad: 6s] (8)
[1776618346257 ms]      Consumidor extrae de Prod 3 (Prio 3) valor '2' (5)
[1776618347257 ms]      Consumidor extrae de Prod 3 (Prio 3) valor '2' (6)
[1776618348256 ms] Productor 1 (Prio 1) mete '33' [Caducidad: 12s] (9)
[1776618348257 ms]      Consumidor extrae de Prod 1 (Prio 1) valor '33' (7)
[1776618351258 ms]      (Caducado) Prod 3 (Prio 3) valor '3' tras 15s (Max: 7s)
Resultados (por caducidad):
Suma Fichero 1: 36
Suma Fichero 2: 175
Suma Fichero 3: 4
Suma Total (Solo no caducados): 215
```

En este primer experimento se ve que el consumidor sigue funcionando correctamente, respetando el orden de prioridad, ya que el consumidor siempre retira primero el elemento de más alta prioridad. Además, se aprecia que el cambio de los sleeps afecta principalmente a la caducidad, sobre todo en los hilos de prioridad 2 y 3, dado que hay una pequeña disminución de las sumas de los ficheros respecto a las originales.

Por otra parte, estos son los resultados del apartado b:

```
[1776619110500 ms] Productor 1 (Prio 1) mete '1' [Caducidad: 3s] (1)
[1776619110501 ms]      Consumidor extrae de Prod 1 (Prio 1) valor '1' (1)
[1776619111501 ms] Productor 1 (Prio 1) mete '2' [Caducidad: 2s] (2)
[1776619112500 ms] Productor 2 (Prio 2) mete '2' [Caducidad: 11s] (3)
[1776619112501 ms] Productor 1 (Prio 1) mete '33' [Caducidad: 5s] (4)
[1776619114500 ms] Productor 3 (Prio 3) mete '3' [Caducidad: 1s] (5)
[1776619115500 ms] Productor 2 (Prio 2) mete '32' [Caducidad: 12s] (6)
[1776619115501 ms]      (Caducado) Prod 1 (Prio 1) valor '2' tras 4s (Max: 2s)
[1776619115501 ms]      Consumidor extrae de Prod 1 (Prio 1) valor '33' (3)
[1776619118501 ms] Productor 2 (Prio 2) mete '143' [Caducidad: 9s] (7)
[1776619119500 ms] Productor 3 (Prio 3) mete '2' [Caducidad: 10s] (8)
[1776619121502 ms]      Consumidor extrae de Prod 2 (Prio 2) valor '143' (4)
[1776619124501 ms] Productor 3 (Prio 3) mete '2' [Caducidad: 5s] (9)
[1776619125502 ms]      (Caducado) Prod 2 (Prio 2) valor '2' tras 13s (Max: 11s)
[1776619125502 ms]      Consumidor extrae de Prod 2 (Prio 2) valor '32' (6)
[1776619128503 ms]      Consumidor extrae de Prod 3 (Prio 3) valor '2' (7)
[1776619132503 ms]      (Caducado) Prod 3 (Prio 3) valor '2' tras 13s (Max: 10s)
[1776619132503 ms]      (Caducado) Prod 3 (Prio 3) valor '3' tras 18s (Max: 1s)
Resultados (por caducidad):
Suma Fichero 1: 34
Suma Fichero 2: 175
Suma Fichero 3: 2
Suma Total (Solo no caducados): 211
```

En el segundo apartado, al modificar los tiempos de generación de los elementos y hacerlos fijos, se pueden observar patrones más regulares de producción que en los anteriores apartados. Además, también se da el caso de que estos elementos tengan una mayor caducidad en todas las prioridades, en su mayor parte debido al aumento del tiempo de espera, excepto en la primera prioridad.

5. Conclusiones

Comparando ambos casos, se aprecia que la eliminación de la aleatoriedad de los tiempos tanto de producción como de consumición afectan en el comportamiento general del programa, y generando más estabilidad en el caso b.

Sin embargo, en ambos casos se mantiene el principio de consumo por prioridad, lo cual garantiza que siempre se atienda primero los elementos más prioritarios, aún con las variaciones en los tiempos de producción o con el aumento de caducidades.